



Explore how JOINS work

Hey, welcome back. So far we've checked out a few different tools you can use to aggregate data within spreadsheets. In this video, we'll cover how to use JOIN in SQL to aggregate data in databases. First, I'll tell you a little bit about what a JOIN actually is, and then we'll explore some of the most common JOINS in action. Let's get started.

JOIN is a SQL clause that's used to combine rows from two or more tables based on a related column. Basically, you can think of a JOIN as a SQL version of VLOOKUP which we just covered.

There are four common JOINS data analysts use, inner, left, right, and outer. Here's a handy visualization of what each JOIN actually does. We'll use these to help us understand these functions. JOINS help you combine matching or related columns from different tables. When we learned about relational databases, we refer to these values as primary and foreign keys. Primary keys reference columns in which each value is unique to that table. But that table can have multiple foreign keys which are primary keys in other tables.

For example, in a table about employees, the employee ID is a primary key and the office ID is a foreign key. JOIN use these keys to identify relationships and corresponding values. An inner JOIN is a function that returns records with matching values in both tables. If we think about our tables as a circles of this Venn diagram, then

an inner JOIN would return the records that exist where the tables are overlapping. For the records to appear in the results table, they'll have to be key values in both tables. The records will only merge if there are matches in both tables. When we input JOIN into SQL, it usually defaults to inner JOIN.

A lot of analysts will use JOIN as shorthand instead of typing the whole query. A LEFT JOIN is a function that will return all the records from the left table and only the matching records from the right table. Here's how you can figure out which table is left or right. In English and SQL we read from left to right. The table mentioned first is left and the table mentioned second is right. You can also think of left as a table name to the left of the JOIN statement and right as a table name to the right of the JOIN statement. In this diagram, you'll notice that the entire left table is colored in, and that's the overlap with the right table which shows us that the left table and the records it shares with the right table are being selected.

Each row in the left table appears in the results even if there are no matches in the right table. RIGHT JOIN does the opposite. It will return all records from the right table and only the matching records from the left. You can get the same results if you flip the order of the tables and use a left JOIN. For example, SELECT from table A, LEFT JOIN table B is the same as SELECT from table B, RIGHT JOIN table A. Finally, there's OUTER JOIN. OUTER join combines RIGHT and LEFT JOIN to return all matching records in both tables.

This means it will return all records in both tables. If there are records in one table without a match, it'll create a record with no values for the other table. Using JOINS can make working with multiple data sources a lot easier and it can make relationships

between tables more clear. Here's an example. Let's say we're working with employee data across multiple departments. We have an employees table and a departments table which both have some columns like department ID. We can use different JOIN clauses to help us put different data from our tables and aggregate it.

Maybe we want to get a list of employees with their department name, excluding any employee without a department ID. Because the department ID record is used in both tables, we can use an INNER JOIN to return a list with only those employees. As a quick reminder, analysts will sometimes just input JOIN for an INNER JOIN but for this example, we'll write it out. To build this query, we'll start with SELECT and AS to tell SQL how we want the columns titled. Then we'll use FROM to tell it where we're getting this data, in this case the employees table. Then we'll input INNER JOIN and the other table we're using, which is departments. We can specify which column and each table will contain the matching JOIN key by writing ON employees.department_id equals departments.department_id.

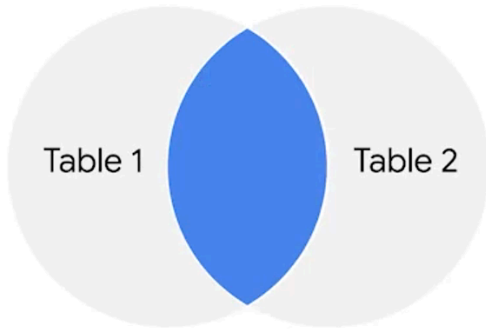
Now, let's run it, and there. Now we've got a list of employee names and department IDs for the employees that have those IDs. But we could use LEFT or RIGHT join to return a list of all employee names and their departments when available. Let's try both really quickly. This will start similar to the last query, we'll put in SELECT AS and FROM again. But this time we'll say LEFT JOIN and use ON like we did with the last query. When we execute the query, we get back this new list with the employee names and departments. But you'll notice there's null values. These are places where the right table which is departments in this case didn't have corresponding values. Let's try RIGHT JOIN just to test it out. This query will be almost the same. Only difference is that we'll use the

RIGHT JOIN clause to return all the rows from the right table, whether they have matching values in the table to the left of the JOIN statement or not. In this case, the right table is departments. Now, let's try out one last JOIN: OUTER.

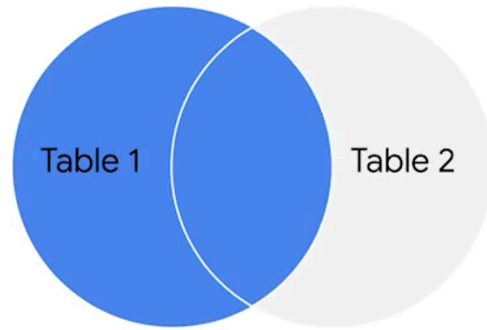
OUTER JOIN will fetch all of the employee names and departments. Again, this query will start a lot like the other ones we've done, we'll use SELECT AS and FROM to choose what data we want and how. We'll grab this from the employees table, and put FULL OUTER JOIN with the departments table to get all of the records from both. We'll also use ON again here. Now we can run this, and we'll get all of the employee names and departments from these tables. There will be nulls in the department.name column, the employee.name column and role column because we've joined columns that don't have matching values, and there. Now you know how JOINS work.

JOINS are super useful when you need to work with data from multiple related tables. They give you a lot of flexibility with how you combine and view that data. If you ever have trouble remembering what INNER, RIGHT, LEFT, or OUTER JOIN do, just think back to our Venn diagram. We'll keep learning about aggregating data in SQL next time. See you soon.

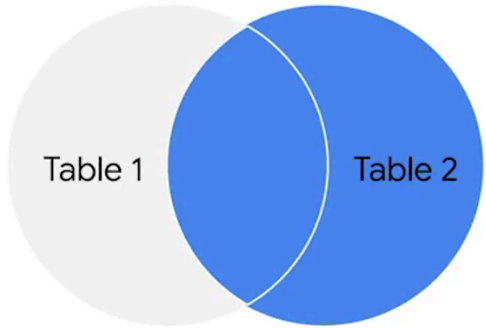
INNER JOIN



LEFT JOIN



RIGHT JOIN



FULL OUTER JOIN

